

Documento de Entrega

Cliente

Flujo de ejecución

El main.cpp del cliente corre un bucle que muestra un menú con tres opciones (listar figuras, pedir una figura, salir). Cada opción crea un socket nuevo, arma la solicitud HTTP y espera la respuesta completa antes de volver a mostrar el menú:

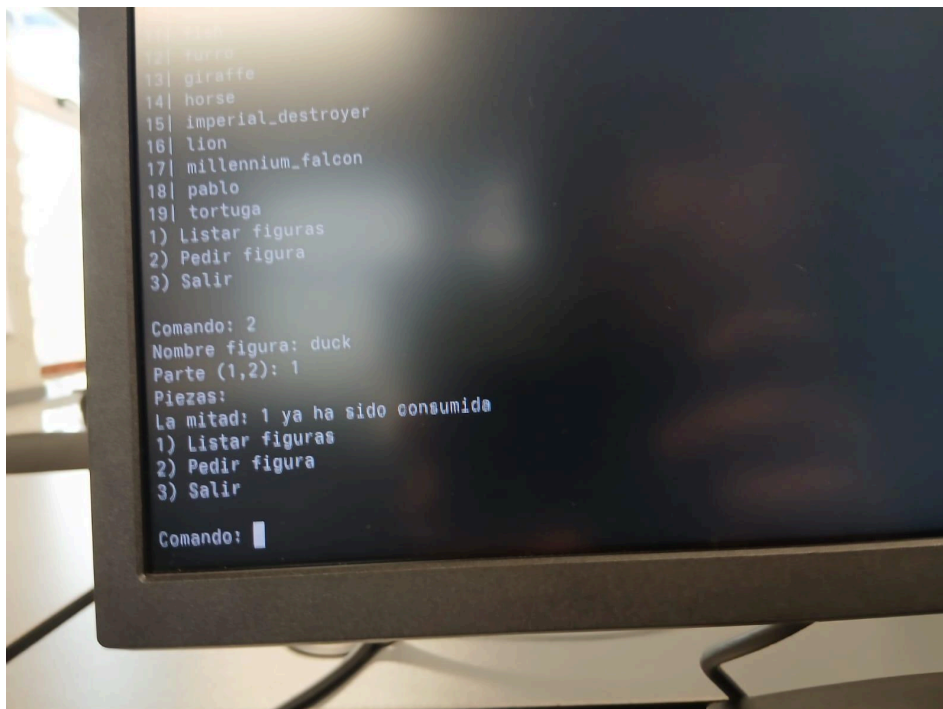
```
while (running) {
    // 1) Listar figuras  2) Pedir figura  3) Salir
    std::getline(std::cin, comando);
    if (comando == "1") {
        VSocket* client = new Socket('s', ipv6);
        cliente.ClientRequestList(client, service, log, host);
        while ((st = client->Read(buffer, MAXBUF-1)) > 0) {
            buffer[st] = 0;
            response += buffer;
        }
        delete client;
    }
    ...
}
```

Client::ClientRequestList / ClientRequestFigure solo arman el texto de la solicitud HTTP y lo escriben en el socket después de conectarse:

```
std::string request =
    "GET /figura/" + figura + "/" + p + " HTTP/1.1\r\n"
    "Host: " + host + "\r\n\r\n";
client->Connect(host, service);
client->Write(request.c_str(), request.length());
```

Ejecución en laboratorio

30/6/2026



Intermediario

Flujo de ejecución

El main() del Intermediario prepara el descubrimiento inicial y luego entra en un ciclo infinito de aceptar conexiones de clientes:

```
Intermediario fork(intermediario, SERVER_HOST, SERVER_PORT);
fork.iniciarDescubrimientoIntermediarios();
fork.getFork()->Bind(PORT);
fork.getFork()->MarkPassive(5);
while (true) {
    VSocket* client = fork.getFork()->AcceptConnection();
    std::thread(&Intermediario::task, &fork, client, ipv6).detach();
}
```

Dentro de task(), se interpreta la ruta HTTP (/figuras, /figura/nombre/mitad, /shutdown). Primero se consulta el servidor local; si este responde 404, se intenta con otros intermediarios por el protocolo TP:

```
std::string response = consultarServidorLocal(rutaServidor, ipv6);
if (response.find("HTTP/1.1 404 Not Found") == 0) {
    std::string piezasRemotas =
        consultarIntermediariosTP(figura, parte, ipv6);
    http = piezasRemotas.empty()
        ? "HTTP/1.1 404 Not Found..."
        : "HTTP/1.1 200 OK..." + piezasRemotas;
}
```

Paralelización

El Intermediario usa varios hilos independientes al mismo tiempo:

- Un hilo nuevo por cada cliente HTTP que se conecta (Intermediario::task), lanzado con std::thread(...).detach().
- Un hilo fijo escuchando paquetes JOIN por UDP de otros intermediarios (escucharIntermediariosTP, puerto 3030).
- Un hilo fijo escuchando HANDSHAKE / INTERMEDIARY_REQUEST por TCP (escucharSolicitudesTP, puerto 3031); dentro de este, por cada conecciones entrante de otro intermediario se lanza además un hilo trabajador (worker) mediante una lambda.

Como varios de estos hilos leen y escriben la tabla de rutas al mismo tiempo, todo acceso a tablaRutas se protege con std::mutex mutexTabla usando std::lock_guard, para evitar condiciones de carrera.

Tablas

tablaRutas es un std::map<std::string, std::list<Ruta>>: la clave es el nombre de la figura y el valor es una lista de rutas (uno o varios servidores/intermediarios que la tienen disponible). Cada Ruta tiene estos campos:

Campo	Tipo	Descripción
host	std::string	IP del servidor o intermediario que ofrece la figura.

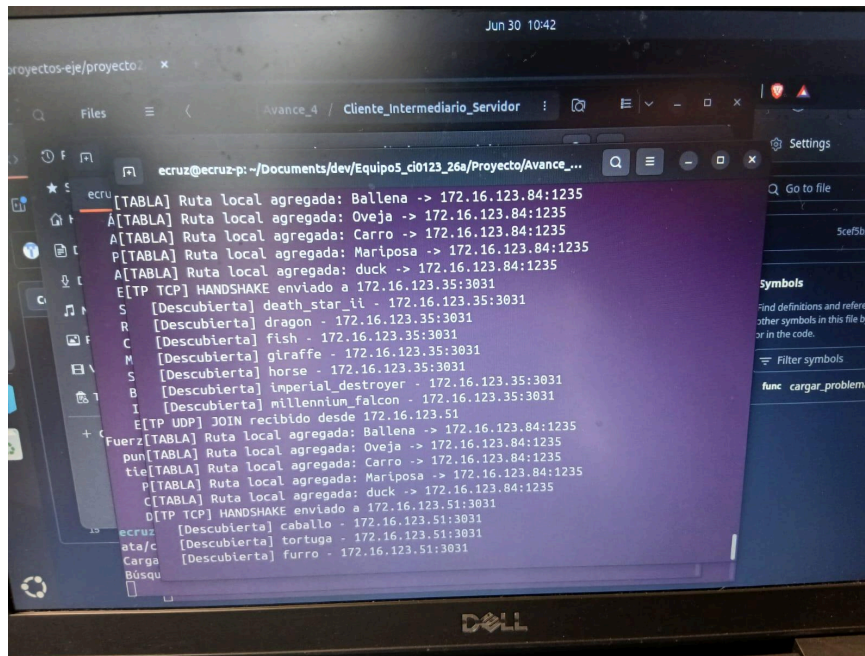
port	std::string	Puerto TCP donde se contacta esa ruta.
activo	bool	Indica si la ruta respondió correctamente la última vez que se usó.
local	bool	true si la ruta apunta al servidor propio del intermediario; false si es de otro intermediario (descubierta por TP).

Manejo de disminución o invalidación

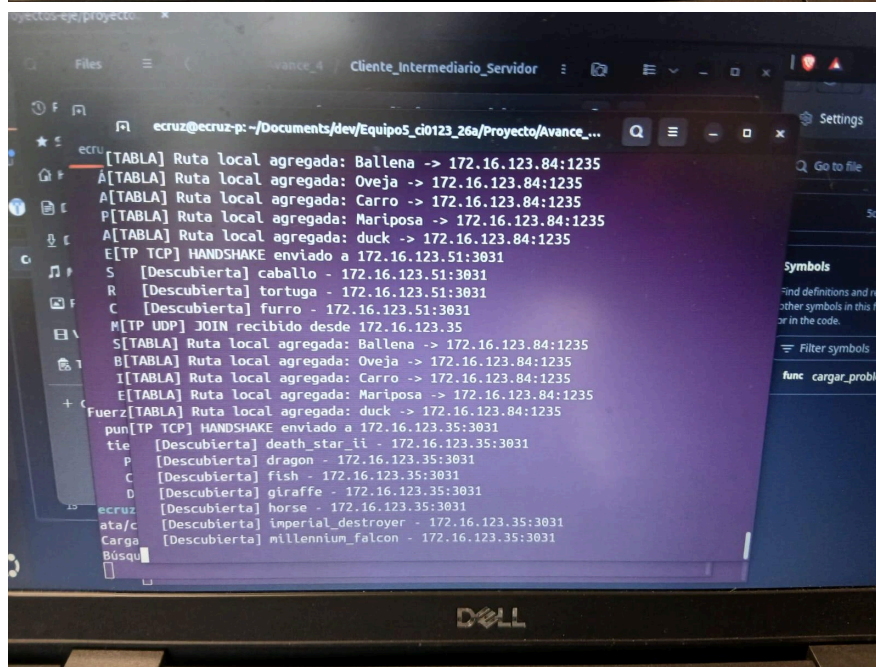
- Si al consultar una ruta remota se recibe `FIGURE_NOT_FOUND`, o falla la conexión (excepción), se marca `ruta.activo = false`.
- `limpiarTablaRutas()` recorre `tablaRutas` y elimina (`remove_if`) todas las rutas con `activo == false`; si una figura se queda sin ninguna ruta, se borra la entrada completa del mapa.
- `eliminarRutasLocales(figura)` quita solo las rutas marcadas como `local == true` de una figura específica, usado al refrescar el estado del servidor propio.
- `actualizarFigurasDesdeServidorLocal()` se llama cada vez que se pide /figuras: primero borra las rutas locales viejas, vuelve a preguntarle al servidor local qué figuras tiene ahora, las re-agrega con `agregarRutaLocal`, y al final llama `limpiarTablaRutas()`.

Ejecución en laboratorio

30/6/2026



```
Jun 30 10:42
ecruz@ecruz-p: ~/Documents/dev/Equipo5_ci0123_26a/Proyecto/Avance_...
[TABLA] Ruta local agregada: Ballena -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: Oveja -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: Carro -> 172.16.123.84:1235
P[TABLA] Ruta local agregada: Mariposa -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: duck -> 172.16.123.84:1235
E[TP TCP] HANDSHAKE enviado a 172.16.123.35:3031
S [Descubierta] death_star_ii - 172.16.123.35:3031
R [Descubierta] dragon - 172.16.123.35:3031
C [Descubierta] fish - 172.16.123.35:3031
M [Descubierta] giraffe - 172.16.123.35:3031
S [Descubierta] horse - 172.16.123.35:3031
B [Descubierta] imperial_destroyer - 172.16.123.35:3031
I [Descubierta] millennium_falcon - 172.16.123.35:3031
E[TP UDP] JOIN recibido desde 172.16.123.51
Fuerz[TABLA] Ruta local agregada: Ballena -> 172.16.123.84:1235
pun[TABLA] Ruta local agregada: Oveja -> 172.16.123.84:1235
tie[TABLA] Ruta local agregada: Carro -> 172.16.123.84:1235
P[TABLA] Ruta local agregada: Mariposa -> 172.16.123.84:1235
c[TABLA] Ruta local agregada: duck -> 172.16.123.84:1235
D[TP TCP] HANDSHAKE enviado a 172.16.123.51:3031
ecruz [Descubierta] caballo - 172.16.123.51:3031
ata/c [Descubierta] tortuga - 172.16.123.51:3031
Carga [Descubierta] furro - 172.16.123.51:3031
Búsqueda
```



```
ecruz@ecruz-p: ~/Documents/dev/Equipo5_ci0123_26a/Proyecto/Avance_...
[TABLA] Ruta local agregada: Ballena -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: Oveja -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: Carro -> 172.16.123.84:1235
P[TABLA] Ruta local agregada: Mariposa -> 172.16.123.84:1235
A[TABLA] Ruta local agregada: duck -> 172.16.123.84:1235
E[TP TCP] HANDSHAKE enviado a 172.16.123.51:3031
S [Descubierta] caballo - 172.16.123.51:3031
R [Descubierta] tortuga - 172.16.123.51:3031
C [Descubierta] furro - 172.16.123.51:3031
M[TP UDP] JOIN recibido desde 172.16.123.35
S[TABLA] Ruta local agregada: Ballena -> 172.16.123.84:1235
B[TABLA] Ruta local agregada: Oveja -> 172.16.123.84:1235
I[TABLA] Ruta local agregada: Carro -> 172.16.123.84:1235
E[TABLA] Ruta local agregada: Mariposa -> 172.16.123.84:1235
Fuerz[TABLA] Ruta local agregada: duck -> 172.16.123.84:1235
pun[TP TCP] HANDSHAKE enviado a 172.16.123.35:3031
tie [Descubierta] death_star_ii - 172.16.123.35:3031
P [Descubierta] dragon - 172.16.123.35:3031
C [Descubierta] fish - 172.16.123.35:3031
D [Descubierta] giraffe - 172.16.123.35:3031
ecruz [Descubierta] horse - 172.16.123.35:3031
ata/c [Descubierta] imperial_destroyer - 172.16.123.35:3031
Carga [Descubierta] millennium_falcon - 172.16.123.35:3031
Búsqueda
```

Servidor

Flujo de ejecución

El main() de ThreadMirrorServer crea el FileSystem una sola vez (variable global fs), abre el socket pasivo, lanza dos hilos fijos de descubrimiento/anuncio, y entra en el ciclo de aceptar clientes:

```
FileSystem fs("filesystem.dat");
s1->Bind(PORT);
s1->MarkPassive(5);
std::thread(escucharDescubrimientoIntermediario).detach();
std::thread(anunciarServidor).detach();
for (;;) {
    client = s1->AcceptConnection();
    std::thread worker(task, client);
    worker.detach();
}
```

task() lee la solicitud HTTP y, según la ruta, consulta el FileSystem. Por ejemplo, para /figuras protege el acceso con el mutex antes de leer:

```
} else if (request.find("GET /figuras") != std::string::npos) {
    std::vector<std::string> figs;
    {
        std::lock_guard<std::mutex> lock(fsMutex);
        figs = fs.getFiguras();
    }
    response = "HTTP/1.1 200 OK...\r\n\r\n";
    for (auto& f : figs) response += f + "\n";
}
```

El hilo anunciarServidor hace broadcast UDP cada 5 segundos para que los intermediarios sepan que el servidor existe:

```
void anunciarServidor() {
    while (true) {
        std::string mensaje = "SERVER_JOIN|port=" + std::to_string(PORT);
        udp->Broadcast(mensaje.c_str(), mensaje.length(),
                      BROADCAST_ISLA, DISCOVERY_PORT_INTERMEDIARIO);
        sleep(5);
    }
}
```

Paralelización

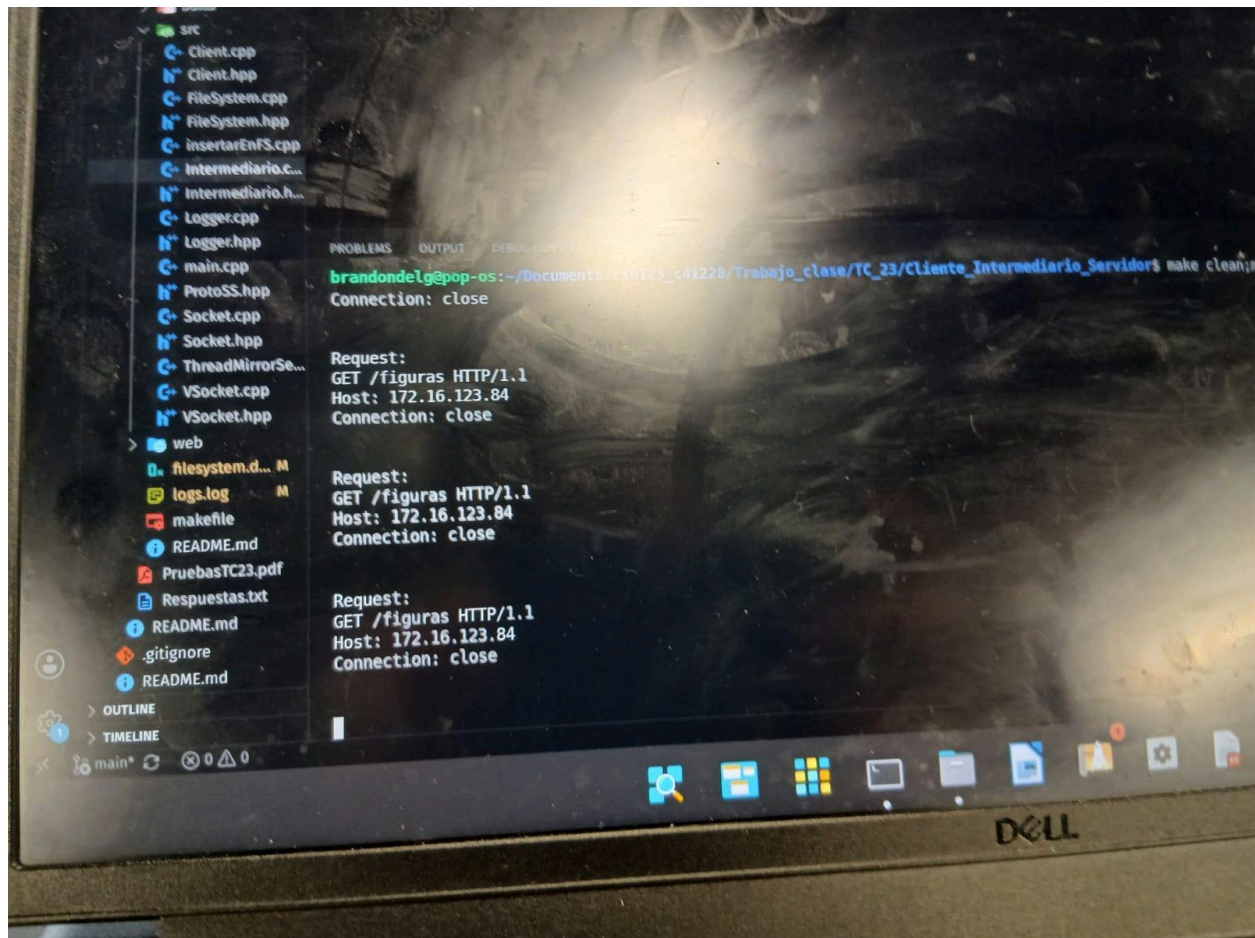
- Un hilo nuevo por cada cliente/intermediario que se conecta al puerto del servidor (task), igual que el Intermediario.
- Un hilo fijo escuchando descubrimiento UDP (escucharDescubrimientoIntermediario, puerto 9093), que responde a los intermediarios que anuncian su presencia.
- Un hilo fijo que anuncia periódicamente el servidor por broadcast UDP (anunciarServidor).

Todo acceso al FileSystem compartido (fs) se protege con std::mutex fsMutex mediante std::lock_guard, porque varios hilos de task() pueden llamar a getFiguras() o getPiezas() al mismo tiempo. El Servidor no mantiene tablas propias ni decide sobre

disminución o invalidación de figuras; delega todo eso al FileSystem, descrito a continuación.

Ejecución en laboratorio

30/6/2026



FileSystem

Flujo de ejecución

El FileSystem no es un proceso ni tiene hilos propios: es una clase que se instancia una sola vez en el Servidor y se usa de forma sincronizada desde afuera con fsMutex. Al construirse, si el archivo filesystem.dat no existe, se llama a init() para crear el SuperBlock, el bitmap y el bloque raíz del directorio:

```
FileSystem::FileSystem(const std::string& filename) {
    disk.open(filename, ios::in | ios::out | ios::binary);
    if (!disk.is_open()) {
        disk.open(filename, ios::out | ios::binary);
        disk.close();
        disk.open(filename, ios::in | ios::out | ios::binary);
        init();
    }
}
```

Cada operación (crearFigura, buscarFigura, getPiezas, borrarFigura, etc.) hace lecturas y escrituras directas sobre el archivo en disco, bloque por bloque. El FileSystem no implementa ninguna sincronización propia (no tiene mutex interno); depende de que el Servidor tome fsMutex antes de llamar a cualquiera de sus métodos:

```
void FileSystem::readBlock(int blockNum, char* buffer) {
    disk.seekg(blockNum * BLOCK_SIZE);
    disk.read(buffer, BLOCK_SIZE);
}

void FileSystem::writeBlock(int blockNum, const char* buffer) {
    disk.seekp(blockNum * BLOCK_SIZE);
    disk.write(buffer, BLOCK_SIZE);
    disk.flush();
}
```

Estructura

SuperBlock (bloque 0) — configuración general del "disco":

Campo	Tipo	Descripción
blockSize	int	Tamaño de cada bloque en bytes (256).
totalBlocks	int	Cantidad total de bloques del archivo.
rootDirBlock	int	Número de bloque donde empieza el directorio raíz (2).
bitmapBlock	int	Número de bloque donde está el bitmap de bloques libres/ocupados (1).

Bitmap (bloque 1): un arreglo de BLOCK_SIZE bytes; cada byte marca si el bloque correspondiente está libre (0) u ocupado (1).

DirBlock (bloque 2 en adelante, encadenados con nextBlock): cada uno tiene hasta 4 entradas DirEntry:

Campo	Tipo	Descripción
nombre	char[32]	Nombre de la figura.

inodeBlock	int	Número de bloque donde está guardado el Inode de esa figura.
------------	-----	--

Inode: describe dónde están los datos de una figura:

Campo	Tipo	Descripción
size	int	Tamaño en bytes del contenido guardado (texto de piezas).
blocks	int[8]	Hasta 8 números de bloques de datos; -1 si esa posición no se usa.

Manejo de disminución o invalidación de figuras

- consumirPiezas(figura, mitad): resta 1 a la cantidad de cada pieza dentro de la mitad indicada, línea por línea, sin borrar nada mientras la cantidad no llegue a 0.
- consumirMitad(figura, mitad): elimina por completo el bloque de texto de una mitad (todas sus líneas). Si al hacerlo ya no queda ninguna mitad "1" ni "2" en el contenido, borra la figura entera llamando a borrarFigura.
- getPiezas() llama automáticamente a consumirMitad() después de leer y devolver las piezas de esa mitad; por eso cada mitad solo puede consultarse una vez — en la siguiente consulta ya aparece como consumida.
- borrarFigura(nombre): libera con freeBlock todos los bloques de datos del Inode y el propio bloque del Inode, y limpia (memset a 0) la entrada correspondiente en el DirBlock.
- freeBlock(blockNum): pone en 0 el byte correspondiente del bitmap para marcar ese bloque como disponible de nuevo.

Diagrama visual

